Vôlei — Documentação para Desenvolvedores

Versão 1.0.1-alpha

Visão geral e arquitetura

O app é um site estático: não existe servidor próprio. O navegador baixa as páginas do GitHub Pages e conversa direto com o Supabase, que guarda os dados e as fotos. Toda a lógica roda no JavaScript do navegador.

```
Navegador (celular ou computador)
   │   roda o JavaScript do app
   │   lembra quem é você no próprio navegador
   │
   ├── baixa as páginas ──► GitHub Pages (repositório aron-dan/volei)
   │                        HTML, CSS e JavaScript estáticos, sem servidor
   │                        publica a branch main
   │                        git push na main atualiza o site em cerca de 1 minuto
   │
   └── lê e grava direto ─► Supabase (projeto "volei"), com a chave pública
                            ├── Banco Postgres: players, ratings, partida, pins
                            │                   view player_stats (médias)
                            └── Storage: bucket fotos (leitura pública)
```

O código que você edita fica no GitHub. Tudo o que muda no dia a dia (jogadores, notas, times, PINs e fotos) fica no Supabase.

Decisões de projeto

O projeto é um hobby de um grupo de amigos, e as escolhas priorizam simplicidade e custo zero em vez de segurança forte.

Decisão	Por que	O que se perde
Site estático no GitHub Pages	Sem servidor para manter, sem DNS e sem custo. Publicar é um `git push`.	Nenhum código roda no servidor
Supabase para dados e fotos	O Pages não guarda dados compartilhados. O Supabase dá Postgres e armazenamento de arquivos no plano gratuito.	Dependência de um serviço externo que pode pausar o projeto
Sem build: HTML, CSS e JS puros	Edita o arquivo e publica, sem npm nem bundler. O `supabase-js` vem de uma CDN.	Sem módulos nem minificação, e o código se repete entre páginas
Entrada por PIN de 4 números (1.0.1)	Evitar que alguém entre na conta errada sem querer. Sem criptografia e sem login de verdade, por decisão do dono. A autenticação real fica para uma versão futura.	O PIN é texto puro numa tabela legível pela API; quem ler o código consegue agir como qualquer jogador
PIN em tabela separada (`pins`)	A view `player_stats` usa `p.*`; uma coluna nova em `players` iria para todas as telas. Em tabela própria, o PIN não trafega junto com as cartas.	Mais uma tabela, e jogador novo precisa de uma linha nela
PIN conferido no banco, sem baixá-lo	`checarPin` pergunta "existe linha com este jogador e este PIN?", então o navegador nunca recebe o PIN guardado.	Continua legível por quem consultar a API direto
`/maneger` aberto, sem senha	Grupo de amigos. Só quem recebe o link o encontra.	Quem descobrir o endereço consegue sortear, cadastrar, remover e resetar PINs
Nota da carta = média das avaliações do app	As notas da planilha original foram dadas "de cabeça" e ficaram de fora.	Carta instável com poucas avaliações
Habilidade sem nota aparece como 3	Todo mundo começa igual até a galera avaliar.	Mostra 3 mesmo sem avaliação nenhuma (o contador `avaliacoes` mostra 0)
Todas as cartas no mesmo estilo	Para ninguém se sentir menosprezado: o grupo é de resenha, não de competição. Não há ouro, prata nem bronze.	Nenhum destaque visual por nível
Recorte de foto	A detecção automática de rosto não achou o rosto nas fotos de	O jogador precisa enquadrar a foto

manual (arrastar e zoom)	teste.	
Vulgo em destaque, nome pequeno	A galera se conhece pelo apelido.	Nenhum
Mobile-first e visual de jogo de futebol	O uso é quase todo pelo celular, no estilo Winning Eleven / bomba patch.	O `/maneger` no celular é mais apertado que no computador

Estrutura do repositório

São 8 arquivos de código, cerca de 1000 linhas no total, mais a documentação. Cada página é uma pasta com um `index.html`, o que dá endereços limpos no GitHub Pages (`/player/`, `/maneger/`). A branch é `main`, e o repositório é público porque o Pages gratuito exige isso.

Cada página carrega, nesta ordem: o `supabase-js` (CDN), `assets/app.js` e depois o script da própria página, que fica inline no HTML. Só `player/index.html` carrega também `assets/foto.js`.

Arquivo	Linhas	Papel
`index.html`	88	Tela inicial: o jogador escolhe a própria carta e entra com o PIN
`player/index.html`	160	Minha carta: editar vulgo, característica e foto; confirmar presença; links para as outras telas
`player/avaliar/index.html`	149	Lista de cartas e painel de notas de 1 a 5
`player/partida/index.html`	74	Times publicados para a próxima partida
`maneger/index.html`	256	Presença, sorteio, publicação, cadastro de jogadores e reset de PIN
`assets/app.js`	75	Cliente Supabase, versão, identidade do jogador, PIN e desenho das cartas
`assets/foto.js`	123	Recorte da foto no canvas e envio ao Storage
`assets/style.css`	88	Visual comum, cartas, lista de cartas e tela de PIN
`README.md`, `CHANGELOG.md`, `banco.sql`, `docs/`		Documentação e o SQL completo do banco

Banco de dados

Tudo vive no projeto "volei" do Supabase: quatro tabelas, uma visão e um bucket de fotos. O app lê quase sempre a visão `player_stats`, e só grava nas tabelas. O arquivo `banco.sql` tem o SQL completo e pronto para rodar num projeto novo.

Objeto	Para que serve	Pontos de atenção
`players`	Um registro por jogador: nome, vulgo, característica, foto e se vai jogar	`nome` pode ser vazio (existe jogador só com vulgo). `caracteristica` é texto livre; os valores vêm da tela
`ratings`	Uma avaliação de um jogador por outro, com 5 notas de 1 a 5	`unique (avaliador_id, avaliado_id)`: refazer uma avaliação é um `upsert`. As notas podem ser vazias. Apagar um jogador apaga suas avaliações (`on delete cascade`)
`partida`	A partida publicada: uma única linha (`id = 1`)	`times` é uma lista de listas de ids de jogadores, em `jsonb`. Publicar sobrescreve
`pins`	O PIN de cada jogador: `player_id`, `pin` (texto de 4 dígitos) e `definido`	`definido = false` obriga o jogador a criar um PIN novo no próximo acesso. Apagar o jogador apaga a linha (`on delete cascade`). **Jogador sem linha aqui não consegue entrar**
`player_stats` (visão)	Os dados de `players` mais as médias das 5 notas, o total e a contagem de avaliações	Média sem nota vira 3. O `p.*` é expandido na criação: ao incluir uma coluna em `players`, recrie a visão. **Não inclui o PIN**
Bucket `fotos`	Fotos dos jogadores, público para leitura	Arquivo `<id do jogador>-<horário>.jpg`, um novo a cada troca

Estado atual, reconstruído dos comandos executados no SQL Editor (a versão completa e ordenada está em `banco.sql`):

```sql
create table players (
  id uuid primary key default gen_random_uuid(),
  nome text,
  vulgo text,
  caracteristica text,
  foto_url text,
  vai_jogar boolean default false,
  created_at timestamptz default now()
);

create table ratings (
  id uuid primary key default gen_random_uuid(),
  avaliador_id uuid references players(id) on delete cascade,
  avaliado_id uuid references players(id) on delete cascade,
  ataque int check (ataque between 1 and 5),
  recepcao int check (recepcao between 1 and 5),
  levantamento int check (levantamento between 1 and 5),
  saque int check (saque between 1 and 5),
  mobilidade int check (mobilidade between 1 and 5),
  unique (avaliador_id, avaliado_id)
);

create table partida (
  id int primary key default 1 check (id = 1),
  times jsonb not null default '[]',
  atualizado timestamptz default now()
);
insert into partida (id) values (1);

-- 1.0.1-alpha
create table pins (
  player_id uuid primary key references players(id) on delete cascade,
  pin text not null default '0000',
  definido boolean not null default false
);

create view player_stats with (security_invoker = true) as
select p.*,
  round(coalesce(avg(r.ataque),3),1) as ataque,
  round(coalesce(avg(r.recepcao),3),1) as recepcao,
  round(coalesce(avg(r.levantamento),3),1) as levantamento,
  round(coalesce(avg(r.saque),3),1) as saque,
  round(coalesce(avg(r.mobilidade),3),1) as mobilidade,
  round(coalesce(avg(r.ataque),3)+coalesce(avg(r.recepcao),3)+coalesce(avg(r.levantamento),3)
   +coalesce(avg(r.saque),3)+coalesce(avg(r.mobilidade),3),1) as total,
  count(r.id) as avaliacoes
from players p left join ratings r on r.avaliado_id = p.id
group by p.id;

-- acesso totalmente aberto (ver Segurança)
alter table players enable row level security;
alter table ratings enable row level security;
alter table partida enable row level security;
alter table pins enable row level security;
create policy "aberto" on players for all using (true) with check (true);
create policy "aberto" on ratings for all using (true) with check (true);
create policy "aberto" on partida for all using (true) with check (true);
create policy "aberto" on pins for all using (true) with check (true);

-- fotos
insert into storage.buckets (id, name, public) values ('fotos','fotos',true);
create policy "fotos leitura" on storage.objects for select using (bucket_id = 'fotos');
create policy "fotos envio" on storage.objects for insert with check (bucket_id = 'fotos');
create policy "fotos troca" on storage.objects for update using (bucket_id = 'fotos');
create policy "fotos apagar" on storage.objects for delete using (bucket_id = 'fotos');
```

Os arquivos de foto não podem ser apagados por SQL no Supabase; use a API (o app faz isso) ou o painel Storage.

Código compartilhado e identidade do jogador

`assets/app.js` é carregado por todas as páginas e define tudo o que elas dividem. Como as páginas montam HTML com `innerHTML`, todo texto que vem do banco passa por `esc()`; mantenha isso ao criar telas novas.

Item	O que faz
`VERSAO`	Texto da versão (`'1.0.1-alpha'`), mostrado no rodapé da tela inicial
`sb`	Cliente do Supabase, criado com a URL do projeto e a chave publishable (pública por natureza), escritas direto no arquivo

`POS`, `SIL`	Sigla de cada característica (ATA, LEV, VER, REC) e a silhueta usada quando não há foto
`esc(s)`	Escapa `& < > " '` antes de entrar em `innerHTML`
`nomeExibido(p)`	Vulgo, senão nome, senão `???`
`sc(v)`	Converte a média de 1 a 5 para a escala de 0 a 99 (`round(v * 19.8)`)
`cardHTML(p)`, `cardMini(p, s)`	Carta em tamanho real e reduzida (ver Cartas)
`meuId()`, `escolherId(id)`, `limparId()`	Identidade do jogador (abaixo)
`pinValido(p)`	Confere se o texto tem exatamente 4 dígitos
`checarPin(id, pin)`	Pergunta ao banco se existe linha com esse jogador e esse PIN. Devolve `null` (errado) ou `{ definido }`
`trocarPin(id, novo)`	Grava o PIN novo e marca `definido = true`. Devolve `true`/`false`

Como o app sabe quem é quem

Não há login de verdade: o jogador escolhe a carta na tela inicial, digita o PIN e o id dele passa a identificá-lo.

Dois pares de chaves guardam a identidade, cada um em `sessionStorage` (a aba) e em `localStorage` (o navegador):

- `jogador_id`: quem é a pessoa.
- `jogador_ok`: a **prova** de que esse id passou pelo PIN. Só `escolherId()` grava, e ela só roda depois do PIN certo.

`meuId()` procura o id, nesta ordem: o parâmetro `?id=` da URL, o `sessionStorage` da aba e o `localStorage` do navegador. **Mas só devolve o id se ele bater com uma `jogador_ok`** (da aba ou do navegador). Sem prova, devolve `null`, e as páginas mandam a pessoa de volta para a tela inicial. Por isso abrir `player/?id=...` direto não pula o PIN. `limparId()` (botão "Não sou eu") apaga as quatro chaves.

Por que três níveis para achar o id: só com `localStorage`, duas pessoas testando no mesmo navegador sobrescreviam a escolha uma da outra, e quem atualizava a página caía no perfil errado. O `?id=` faz o link carregar a identidade, e o `sessionStorage` isola as abas. Os botões de `player/index.html` já levam `?id=` para Avaliar e Próxima partida, e essas páginas apontam o "voltar" de volta com o id.

Limites conhecidos:

- Na mesma aba, a escolha mais recente vale.
- Uma aba nova no mesmo navegador herda o último jogador que entrou (pelo `localStorage`).
- Quem já tinha entrado antes da 1.0.1 não tem `jogador_ok`, então todos passaram pelo PIN uma vez.
- Se o armazenamento do navegador falhar, `meuId()` devolve `null` e a pessoa não entra.

Páginas e fluxos

Cada página busca os dados ao abrir, desenha tudo com `innerHTML` e grava direto no Supabase quando o usuário age. Não há estado compartilhado entre páginas além do id do jogador.

Página	Lê	Grava
`index.html`	`player_stats` (todos) e desenha uma carta reduzida por jogador. A tela de PIN lê `pins` só pela pergunta de `checarPin`	`pins` (`trocarPin`, no primeiro acesso ou após reset). Ao entrar chama `escolherId` e vai para `player/?id=`
`player/index.html`	`player_stats` do próprio jogador	`players`: vulgo e característica (botão Salvar), `vai_jogar` (Vou jogar! / Não vou) e `foto_url` (via `foto.js`)
`player/avaliar/index.html`	`player_stats` (todos) e as `ratings` em que o jogador é o avaliador	`ratings` por upsert em (`avaliador_id`, `avaliado_id`); nota repetida limpa o campo (grava `null`); exige ao menos uma nota
`player/partida/index.html`	`partida` (times e horário) e `player_stats`; cruza os ids dos times com os jogadores	Nada. Destaca o time e a carta de quem está vendo
`maneger/index.html`	`player_stats`; a cada 20 s e ao	`players` (`vai_jogar` de um ou de todos, insert, update de

	voltar para a aba, só `id` e `vai_jogar` de `players`	nome e vulgo, delete), `pins` (upsert no cadastro e no Reset PIN), `partida` (publicar) e Storage (apaga a foto ao remover um jogador)

Detalhes que costumam pegar:

- **Tela de PIN (`index.html`):** é uma camada sobre a lista com três modos: `entrar` (digita o PIN), `novo` (primeiro acesso ou reset) e `confirmar` (repete o novo). O PIN novo não pode ser igual ao que a pessoa acabou de usar para entrar.
- **Textos das notas:** as descrições de cada nível (1 a 5) de cada habilidade estão escritas no objeto `DESC`, dentro de `player/avaliar/index.html`. Para mudar um texto, edite lá.
- **Atualização automática do /maneger:** não roda se houver campo de texto em foco, texto digitado no cadastro ou linha em edição, para não apagar o que a pessoa escreve. Quando muda alguém, a tela é redesenhada e o sorteio na tela é mantido.
- **PIN de reset (`/maneger`):** vive na variável `pinReset` (começa em `'0000'`), alimentada pelo campo acima da lista do cadastro. Não é gravada em lugar nenhum, então volta a `0000` ao recarregar a página.
- **Reset PIN e cadastro usam `upsert`** em `pins`: criam a linha se ela não existir e atualizam se existir. Por isso o reset também conserta um jogador que ficou sem linha (por exemplo, criado direto pelo SQL).
- Mudar presença no `/maneger` zera o sorteio da tela (`resultado = null`), porque os times deixam de corresponder aos confirmados.
- **Remover jogador:** o `delete` em `players` apaga as avaliações e o PIN em cascata, e o código apaga a foto no Storage por conta própria. Uma partida já publicada continua guardando o id antigo; a tela de partida ignora ids que não existem mais.

Cartas

A carta é HTML e CSS puros, montada por `cardHTML(p)` a partir de uma linha de `player_stats`. Os números vêm da escala de 0 a 99: `sc(média) = round(média × 19,8)`, então nota 1 vale 20, nota 3 vale 59 e nota 5 vale 99.

- **Número grande (OVR):** média dos 5 valores já convertidos, arredondada. Aparece `--` se nenhuma habilidade tiver valor.
- **Sigla:** `POS[caracteristica]`, ou `--` se o jogador ainda não escolheu.
- **Foto:** `background-image` na área `.photo`, com máscara que apaga a borda de baixo. Sem foto, usa a silhueta `SIL`.
- **Nome:** o vulgo grande (`.name`), e o nome real pequeno (`.real`) só quando existem os dois.

O formato é um hexágono feito com `clip-path` em duas camadas: `.card` (270 × 400) é a borda em degradê e `.inner`, 4 px para dentro, é a face. Todo o visual da carta vem de três variáveis em `.special` (`--edge`, `--face`, `--ink`), então trocar as cores é mexer só ali. Existe um brilho animado em `.inner::after`.

Carta reduzida

`cardMini(p, s)` não redesenha nada: embrulha a carta inteira em `.cm`, que tem o tamanho final (270 × s por 400 × s), e escala o conteúdo com `transform: scale(var(--s))`. Por isso a carta continua idêntica em qualquer tamanho. Escalas em uso:

Tela	Escala
Escolher jogador e lista de quem vai jogar	0,4
Times no sorteio e na próxima partida	0,5
Lista em Avaliar	0,55
Painel de notas em Avaliar	0,62

Na versão reduzida o brilho animado e a sombra são desligados (`.cm .inner::after` e `.cm .slot`): com 20 cartas na tela, tantas animações juntas poderiam pesar no celular, e foram desligadas por precaução. Textos abaixo de 0,4 ficam pequenos demais, então evite escalas menores.

Atenção: a média do time no `/maneger` usa `round(total / 5 × 19,8)` sobre o `total` da visão, que pode divergir em 1 ponto do OVR da carta por causa dos arredondamentos.

Sorteio de times

A função `sortear(jogadores, tamanho)` está em `maneger/index.html` e roda no navegador. Ela não busca o time perfeito: faz muitas tentativas aleatórias, melhora cada uma trocando jogadores de lugar e sorteia entre as melhores.

1. **Número de times:** `k = max(2, round(n / tamanho))`.
2. **Pontuação do jogador:** o `total` da visão (soma das 5 médias, de 5 a 25). Se vier nulo, usa a média do grupo.
3. **Custo de uma divisão (menor é melhor):** a soma, por time, do quadrado da diferença entre a média do time e a média geral, mais `0,6 ×` a variância de quantos jogadores de cada característica há por time (Levantador, Atacante, Recepção e Versátil, um termo por característica).
4. **25 tentativas:** em cada uma, embaralha, distribui um a um entre os times (os tamanhos diferem em no máximo 1) e faz 1500 trocas aleatórias entre dois times. Mantém a troca se o custo não piorar e desfaz caso contrário.
5. **Escolha final:** entre as tentativas com custo até `mínimo + 0,3`, sorteia uma. Isso faz o "Sortear de novo" dar resultados diferentes, sem aceitar times desiguais.

As trocas preservam o tamanho dos times, então o desequilíbrio de tamanho vem só da divisão inicial. Jogadores sem característica não entram em nenhum dos quatro termos de característica.

Parâmetro	Valor	Efeito de aumentar
Peso das características	0,6	O sorteio se preocupa mais em espalhar levantadores e atacantes do que em igualar a força
Tentativas	25	Resultados melhores e mais lentos
Trocas por tentativa	1500	Cada tentativa chega mais perto do ideal
Tolerância da escolha final	0,3	Mais variedade entre os sorteios, com times um pouco menos equilibrados

O tamanho do time vai de 2 a 12 (botões − e +), e o botão só habilita com 4 ou mais confirmados. A média exibida em cada time é a média dos `total` convertidos para a escala da carta.

Situação atual a conhecer: depois que a visão passou a devolver 3 para habilidade sem nota, `total` nunca vem nulo, então o plano B da média do grupo não é usado. Quem ainda não foi avaliado entra no sorteio como 15 (tudo 3). O aviso abaixo dos times no `/maneger` diz isso: "entra com nota 3 em todas as habilidades".

Foto

`assets/foto.js` cuida de tudo o que envolve a foto e só é carregado pela tela Minha carta. O jogador escolhe uma imagem, enquadra o rosto numa moldura e só então a foto vai ao Supabase.

1. **Leitura:** `createImageBitmap(file)` respeita a rotação das fotos de celular. A imagem é reduzida para no máximo 3200 px no maior lado e guardada em um canvas.
2. **Recorte (`abrirRecorte`):** abre uma tela cheia com a moldura de 255 × 285 e um círculo-guia tracejado. Um dedo arrasta, dois dedos fazem pinça, e a roda do mouse e o controle deslizante dão zoom. O zoom vai de 1× a 12× numa escala exponencial, para o controle ser suave perto e longe. A imagem sempre cobre a moldura e começa alinhada ao topo, onde ficam os rostos.
3. **Saída:** ao tocar em **Usar foto**, desenha o enquadramento num canvas de 340 × 380 (mesma proporção da área `.photo` da carta) e gera um JPEG com qualidade 0,88. Cancelar devolve `null`.
4. **Envio (`enviarFoto`):** sobe o arquivo para o bucket `fotos` com nome novo `<id>-<horario>.jpg`, lê a URL pública, grava em `players.foto_url` (com `?v=` para furar cache) e, por fim, apaga o arquivo da foto anterior.

Por que ficou assim:

- **Recorte manual:** a primeira versão detectava o rosto automaticamente (modelo de detecção de rosto do MediaPipe), mas não achou o rosto nas fotos de teste, e foi removida. Se alguém quiser tentar de novo, o ponto de encaixe é `abrirRecorte`: o detector só precisaria sugerir a posição e o zoom iniciais.
- **Nome novo a cada envio:** regravar o mesmo nome (`upsert`) deu erro em quem já tinha foto. O nome novo evita isso, permite cache longo (`cacheControl` de um ano) e deixa a troca aparecer na hora.
- **Limpeza:** a foto antiga é apagada ao trocar, e a foto de um jogador removido é apagada pelo /maneger. Se o app falhar entre o envio e a limpeza, sobra um arquivo órfão no bucket; vale olhar o painel Storage de vez em

quando.

O fim da função grava a foto no banco só depois do upload, então um erro no meio nunca deixa o jogador com uma URL quebrada.

Deploy, cache e ambiente

Não há build nem pipeline: o que está na branch `main` é o que vai ao ar.

1. Clone `https://github.com/aron-dan/volei`.
2. Edite os arquivos. Para testar, sirva a pasta com qualquer servidor estático (por exemplo `python3 -m http.server`) e abra no navegador. O app usa o Supabase de produção, então testes gravam dados reais.
3. Faça o commit e o `git push` na `main`. O GitHub Pages (configurado em Settings, Pages, "Deploy from a branch", `main`, raiz) publica em cerca de 1 minuto.

Para testar uma mudança **sem publicar**, trabalhe numa branch separada (`git checkout -b nome`) e só faça o merge na `main` quando estiver pronto: o Pages só lê a `main`.

Versionamento de cache

O navegador guarda CSS e JS e, no celular, também guardou HTML antigo: depois de vários deploys, jogadores viram telas desatualizadas. Para isso, os arquivos de `assets/` são chamados com um número de versão na URL.

Regra: ao alterar um desses arquivos, aumente o número em todas as páginas que o carregam, no mesmo commit. Sem isso, parte dos jogadores continua com a versão antiga. Para uma página HTML que não muda de nome, quem vê a versão velha resolve fechando a aba e abrindo de novo.

Os números não são iguais entre os arquivos porque cada um subiu só quando foi alterado.

Arquivo	Versão atual	Carregado por
`assets/app.js`	`?v=11`	todas as 5 páginas
`assets/style.css`	`?v=11`	todas as 5 páginas
`assets/foto.js`	`?v=7`	`player/index.html`

Não confunda o `?v=` (cache dos arquivos) com a constante `VERSAO` em `app.js` (versão do app, a que aparece no rodapé). O passo a passo de uma versão nova está no CHANGELOG.

Ambiente

- **Supabase:** projeto "volei". A URL e a chave publishable estão no começo de `assets/app.js`. Mudar de projeto significa trocar essas duas linhas e recriar o banco com o `banco.sql`.
- **Fontes:** Bebas Neue e Oswald, pelo Google Fonts.
- **Biblioteca:** `@supabase/supabase-js@2` pela CDN jsDelivr, sem versão fixa de patch. Se uma atualização quebrar algo, fixe a versão.
- **Mensagens de commit:** curtas, em português.

Segurança e limitações conhecidas

A segurança é praticamente só de "quem não tem o link". As regras do banco estão abertas e a chave publishable fica no repositório, que é público. Qualquer pessoa que ler o código consegue falar com o banco sem passar pelas telas. **O PIN da 1.0.1 não muda isso**: ele evita erro de conta, não ataque.

Risco	Situação hoje	Como reduzir
Quem tem o link do `/maneger` altera tudo	Sorteia, publica, cadastra, remove jogadores e reseta PINs	Dar o link só a quem gerencia. Para proteger de verdade: funções RPC do Postgres que exigem uma senha do manager, ou Supabase Auth
PINs legíveis pela API	A tabela `pins` guarda o PIN em texto puro e a política é aberta: quem consultar a API direto lê todos os PINs	Guardar só um hash e conferir por função RPC (`security definer`), tirando a leitura direta da tabela; ou Supabase Auth
Qualquer um age	A identidade é só o id guardado no navegador,	Supabase Auth e regras que comparem `auth.uid()`

como qualquer jogador	liberado por um PIN de 4 dígitos (10 mil combinações, sem limite de tentativas)	com o dono do registro
Notas individuais não são secretas no banco	As telas só mostram médias, mas a tabela `ratings` é legível pela API	Tirar a leitura de `ratings` do acesso anônimo e expor só a visão de médias, com uma função para gravar a nota
Auto-avaliação e avaliação em nome de outro	A tela esconde o próprio jogador, mas o banco aceita	Uma regra no banco que impeça `avaliador_id = avaliado_id`, e login para o resto
Envio de arquivos ao bucket	Qualquer arquivo, até o limite do plano, pela chave pública	Definir no painel do bucket um limite de tamanho e só aceitar imagens
Perda de dados	Plano gratuito, sem rotina de cópia	Exportar as tabelas de vez em quando pelo painel
Projeto pausado por inatividade	O plano gratuito pode pausar o projeto	Retomar pelo painel do Supabase

Limitações de funcionamento:

- Existe uma única partida publicada; não há histórico de pelada, de times ou de presença.
- Toda página carrega `player_stats` inteira. Para dezenas de jogadores é irrelevante; com centenas, paginação ou colunas específicas.
- A carta usa `color-mix()`, `clip-path` e `mask-image`. Navegadores muito antigos desenham a carta de forma diferente.
- As descrições das notas e os textos da interface estão fixos no código, em português do Brasil.
- O PIN de reset do `/maneger` não é guardado: volta a `0000` ao recarregar.
- Não há testes automatizados. A lógica foi conferida na mão e o sorteio foi exercitado com jogadores simulados.

Como mexer

Receitas para as mudanças mais prováveis. Sempre que mexer em `assets/`, aumente o número de versão nas páginas que o carregam (ver Deploy).

Quero...	**Onde mexer**
Mudar o tamanho padrão do time (hoje 4)	`tamanho = 4` no script de `maneger/index.html`. Os limites 2 e 12 estão nos botões − e +
Mudar as cores da carta	As variáveis `--edge`, `--face` e `--ink` de `.special` em `assets/style.css`
Mudar o texto de uma nota (ex.: o que é "3" em saque)	O objeto `DESC` em `player/avaliar/index.html`
Mudar a nota inicial (hoje 3)	O `coalesce(..., 3)` na visão `player_stats` (SQL). Depois da mudança, nenhum código do site precisa mudar
Mudar a escala da carta (0 a 99)	`sc()` em `assets/app.js` e `ovrDe()` em `maneger/index.html`, que usam o fator 19,8
Mudar a frequência de atualização do /maneger	O `setInterval(..., 20000)` no fim do script de `maneger/index.html`
Mudar o PIN de reset padrão (hoje `0000`)	`let pinReset = '0000'` no script de `maneger/index.html`. O padrão da coluna `pin` (SQL) também é `'0000'`
Mudar a quantidade de dígitos do PIN	`pinValido()` em `assets/app.js`, o `maxlength` dos campos em `index.html` e `maneger/index.html`, e o `inputmode` se precisar
Mudar os textos da tela de PIN	O script e o bloco `.pinbox` em `index.html`; o visual em `assets/style.css` (seção "tela de PIN")
Adicionar uma característica (ex.: Bloqueador)	`CARACTERISTICAS` em `player/index.html`, `POS` em `assets/app.js` e `TIPOS` em `maneger/index.html` (para o sorteio espalhá-la)
Adicionar uma habilidade (ex.: Bloqueio)	Coluna em `ratings` com o `check`; recriar `player_stats` com a média e somar ao `total`; `HAB` e `DESC` em avaliar; o mapa `v` em `cardHTML`; e as contas por 5 em `ovrDe` e na média da carta

Cadastrar um jogador	Pela seção Cadastro de jogadores do `/maneger`, sem tocar no código
Resetar o PIN de um jogador	Botão **Reset PIN** no `/maneger`
Mandar todo mundo recriar o PIN	`update pins set pin = '0000', definido = false;` no SQL Editor
Começar uma nova temporada	O SQL abaixo e apagar os arquivos de foto no painel Storage

```
-- zera avaliações, fotos, presença e a partida publicada
delete from ratings;
update players set foto_url = null, vai_jogar = false;
update partida set times = '[]', atualizado = now() where id = 1;
```

Ideias para a próxima versão

Nada disso foi combinado; são caminhos naturais, em ordem de custo:

1. Limitar o tamanho e o tipo de arquivo do bucket de fotos (só configuração no painel).
2. Guardar o PIN de reset do `/maneger` (por exemplo, no `localStorage`), para não voltar a `0000` ao recarregar.
3. Atualização em tempo real do `/maneger` com Realtime do Supabase, no lugar da consulta a cada 20 segundos.
4. Instalar o app na tela inicial como PWA (manifesto e ícone).
5. Senha do manager por funções RPC, e PIN guardado como hash (ver Segurança).
6. Autenticação de verdade com Supabase Auth, que substitui o PIN.
7. Histórico de partidas, para guardar times e presença de cada pelada.
8. Limpar o código do sorteio: o plano B da média do grupo nunca é usado, já que `total` não vem nulo (ver Sorteio de times).